第6章 图

6.1 图的定义

6.2 图的存储方式

6.3 图的遍历

6.4 图的连通性

6.5 最短路径

6.6 拓扑排序

6.7 关键路径

6.1 图的定义

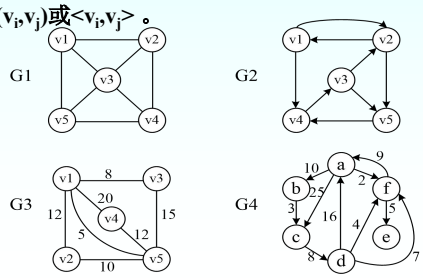
（1）图的定义

•图（Graph）：由顶点集合V和顶点之间关系集合R组成，记为

G=（V，R）

其中：V是顶点的非空有限集合；

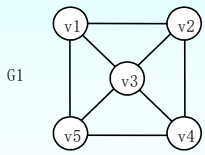
R是两个顶点之间关系的集合，它是顶点的无序或有序对，记作 (v_i,v_j)或$<v_i,v_j>$ 。



图的定义（续）

•无向图：图中顶点之间的关系为无序对。

无序对(v_i,v_j) 称为边E，且(v_i,v_j)= (v_j, v_i)，表示同一条边



在无向图G1中，G1=(V,E)

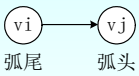
其中，V(G1)={v1,v2,v3,v4,v5}

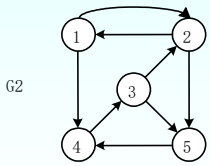
E(G1)={(v1,v2),(v1,v3),(v1,v5),(v2,v3),(v2,v4),(v3,v4),(v3,v5),(v4,v5)}

图的定义（续）

•有向图：图中顶点之间的关系为有序对。

有序对<vi,vj>称为弧A，且<vi,vj>≠<vj , vi >，表示不同的弧。





在有向图G2中，G2=(V,A)

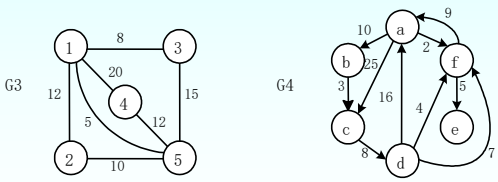
其中，V(G2)={1,2,3,4,5}

A(G2)={<1,2>, <1,4>, <2,1>, <2,5>, <3,2>, <3,5>, <4,3>, <5,4>}

图的定义（续）

•网：G中的每条边（或弧）都带有权值。

网分为有向网和无向网。（见G3和G4）



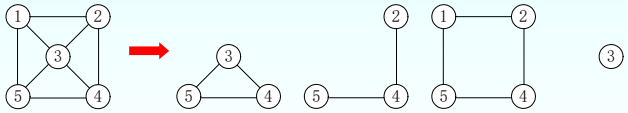
图的基本术语

•子图：如果有两个图G和G’，使得

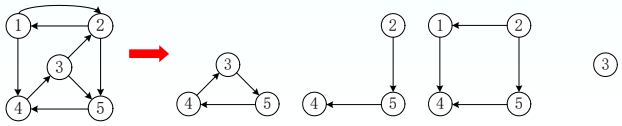
$$V(G')\subseteq V(G)且E(G')\subseteq E(G)$$

则称图G’是G的子图。

G1的某些子图：

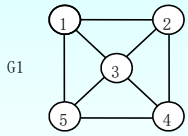


G2的某些子图：



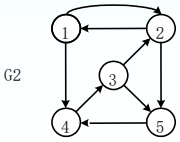
图的基本术语(续)

•在无向图中，与每个顶点相连的边数称为该顶点的度。



例：在G1中，顶点1的度为3，
顶点3的度为4。

•在有向图中，以顶点为头的弧的数目称为该顶点的入度。
以顶点为尾的弧的数目称为该顶点的出度。
顶点的入度与出度之和称为该顶点的度。



例：在G2中，顶点1的入度为1，出度为2；
顶点2的入度为2，出度为2。

度、入度、出度的例题

例:

1)在一个无向图中，所有顶点的度数之和＝所有边数的倍。

2)在一个有向图中，所有顶点的入度之和＝所有顶点的出度之和的_____倍。

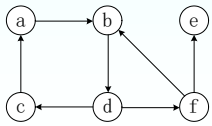
图的基本术语（续）

•路径：在图G中，从顶点v_{i0}到顶点v_{im}的路径是一个顶点序列(v_{i0}, v_{i1}, v_{i2}, …, v_{im-1}, v_{im})。
若此图为无向图，则(v_{ij-1},v_{ij})∈E(G)，(1≤j≤m)
若此图为有向图，则<v_{ij-1},v_{ij}>∈E(G)，(1≤j≤m)

•路径长度：沿此路径上边或弧的数目为路径长度。
对网(有权图)，一般取沿路径上权值之和作为此路径长度。

•简单路径：如果一条路径上的所有顶点都是彼此不同的，则该路径是一条简单路径。

•简单回路：路径的起始和终止在同一顶点上，其他顶点不重合，则该路径是一条简单回路。



1）从顶点a到e的路径为(a,b,d,f,e)，其路径长度为4，此路径是简单路径。

2）路径(b,d,f,b)为一条简单回路，其路径长度为3。

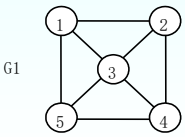
3）路径(a,b,d,f,b)不是一条简单回路，因为其内部存在回路。

图的基本术语（续）

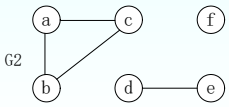
•在无向图中，若从顶点v_i到顶点v_j有路径，则称v_i和v_j是连通的。

•若无向图G中任意两个顶点都连通，则称G是连通图，否则称为非连通图。

•无向图G中的极大连通子图称为G的连通分量。
连通图只有一个连通分量，即本身；
非连通图有多个连通分量。



连通图
有1个连通分量

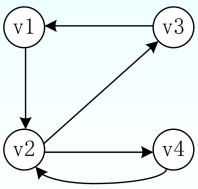


非连通图
有3个连通分量

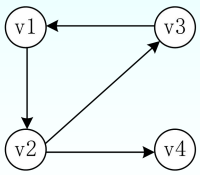
图的基本术语（续）

•若有向图G中任意两个顶点vi和vj都连通（即存在vi到vj的路径，也存在vj到vi的路径），则称G是强连通图。

•有向图G中的极大连通子图称为G的强连通分量。



强连通图
有1个连通分量



非强连通图
有2个连通分量

连通图和连通分量的例题

例1:

1)一个有n个顶点的无向图最少有___ 条边，最多有____条边。
一个有n个顶点的有向图最少有___ 条弧，最多有____条弧。

2)具有n个顶点的无向图至少应有_____条边才能是一个连通图。
具有n个顶点的有向图至少应有__条弧才能是一个强连通图。

例2:

有29条边的无向连通图，至少有___个顶点，至多有___个顶点；有29条边的无向非连通图，至少有____个顶点。

6.2 图的存储方式

6.2.1 顺序存储结构——邻接矩阵

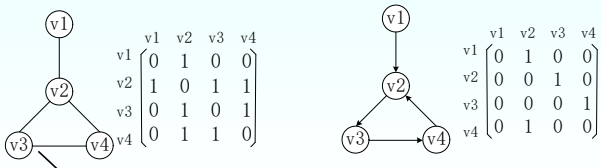
6.2.2 链式存储结构——邻接表

6.2.1 顺序存储——邻接矩阵

假设图G=(V,E)含有n(n>=1)个顶点V={v1,v2,…,vn}，则元素为

$$A[i,j]=\begin{cases}1 & \text{若(vi,vj)或<vi,vj>}\in E(G)\text{，其中}i\neq j\\ 0 & \text{反之}\end{cases}$$

的n*n矩阵A，称为图G的邻接矩阵。



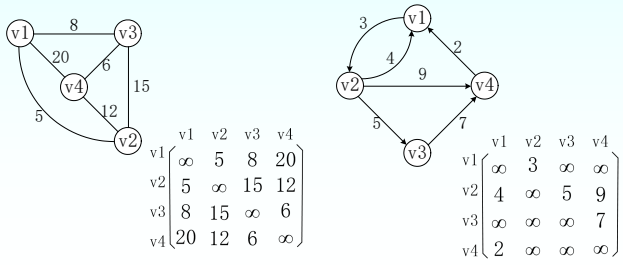
对于无向图，只需存储其上三角(或下三角)矩阵，即只需n(n-1)/2.

顺序存储结构——邻接矩阵（续）

假设网G=(V,E)含有n(n>=1)个顶点V={v1,v2,…,vn}，则元素为

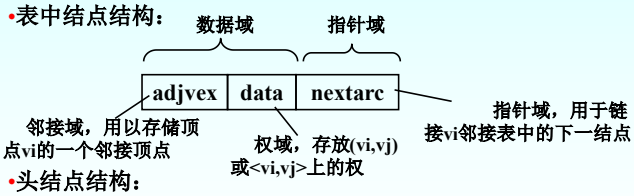
$$A[i,j]=\begin{cases}w_{ij} & \text{若(vi,vj)或<vi,vj>}\in E(G)\text{，且}w_{ij}\text{是}\\ & \text{vi,vj之间的权，其中}i\neq j\\ \infty & \text{反之}\end{cases}$$

的n*n矩阵A，称为网G的邻接矩阵。



6.2.2 链式存储结构——邻接表

- 邻接表：为图中的每个顶点建立一个邻接关系的单链表，并把它们的表头指针用向量存储。
- 表中结点结构：



- 头结点结构：

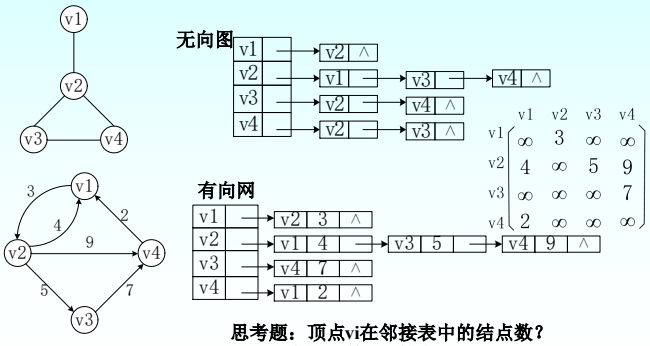
对于每个顶点vi的邻接表，需要设置一个表头结点。



n个顶点对应n个头结点，将这n个头结点用一个向量存储。

链式存储结构——邻接表（续）

例：上面邻接矩阵对应的邻接表



思考题：顶点vi在邻接表中的结点数？
对于无向图，等于vi的度
对于有向图，等于vi的出度

图的存储结构的例题

例：已知有向网采用邻接矩阵存储，其邻接矩阵的三元组表示为(1, 2, 5)，(1, 3, 7)，(2, 4, 3)，(3, 4, 6)，(3, 5, 3)，(4, 5, 4)，(5, 2, 4)，(5, 4, 2)。

（1）写出邻接矩阵

（2）画出有向网

（3）写出邻接表

6.3 图的遍历

6.3.1 深度优先遍历

6.3.2 广度优先遍历

6.3 图的遍历

从图中某一顶点出发访问图中其余顶点，且每个顶点仅被访问一次。

图的遍历 { 深度优先搜索 / 广度优先搜索

为了避免重复访问图中的同一顶点，必须记住每个顶点是否被访问过，为此可设置一个辅助数组visited[1:n](n是顶点数)。

visited[i] { 未访问（false） / 已访问（true）

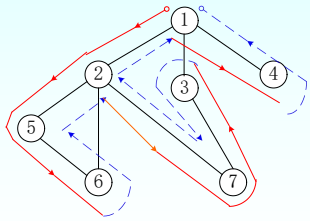
6.3.1 深度优先遍历(DFS, depth-first search)

基本思想：

首先访问一个顶点v_i，并将其标记为已访问，然后从vi的一个未访问过的邻接点出发进行DFS，当v_i的所有邻接点均被访问过时，即退回到上一个顶点v_k，从v_k的另一个未被访问过的邻接点出发进行DFS。

DFS的实现过程

分析以无向图G中的v1作为初始点的深度搜索遍历的过程。



visited[1:7]

7	true
6	true
5	true
4	true
3	true
2	true
1	true

DFS的例题

例：无向图G=(V,E)，其中：V={a,b,c,d,e,f}，

E={(a,b),(a,e),(a,c),(b,e),(c,f),(f,d),(e,d)}，

对该图进行深度优先遍历，得到的顶点序列正确的是（ ）。

A．a,b,e,c,d,f

B．a,c,f,e,b,d

C．a,e,b,c,f,d

D．a,e,d,f,c,b

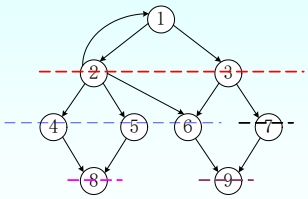
6.3.2 广度优先遍历(BFS, Breadth-first search)

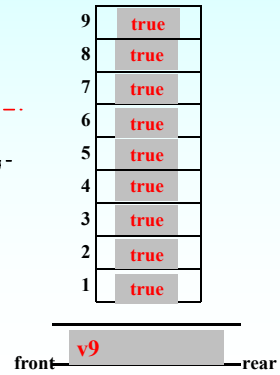
基本思想：

首先访问初始点v_i，并将其标记为已访问，接着访问vi的所有未被访问过的邻接点vi1，vi2，…，vit，并均标记为已访问，然后再按照vi1，vi2，…，vit的次序，访问每一个顶点的所有未被访问过的邻接点，并均标记为已访问过，依次类推，直到图中所有和初始点vi有路径相通的顶点都被访问过为止。

BFS的例题

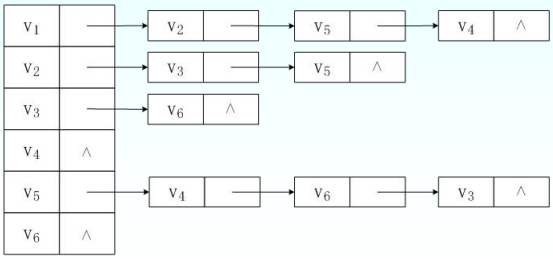
例：分析以有向图G中的v1为出发点，进行BFS遍历的过程。





深度和广度优先搜索的例题

例：已知图G的邻接表如图所示，其从顶点v1出发的深度优先搜索序列为_____，其从顶点v1出发的广度优先搜索序列为____。



深度和广度优先搜索的例题

例：已知图G={V，E}，V={V1，V2，V3，V4，V5，V6，V7，V8，V9，V10}，E={（V1，V2），（V1，V5），（V2，V3），（V2，V8），（V3，V9），（V4，V5），（V4，V9），（V6，V7），（V6，V9），（V6，V10），（V7，V8），（V8，V9），（V9，V10）}。

当它用邻接矩阵和邻接表表示时，分别写出从顶点V1出发按照深度优先和广度优先搜索遍历得到的顶点序列。

假定每个顶点邻接表中的结点是按顶点序号从小到大的次序链接的。

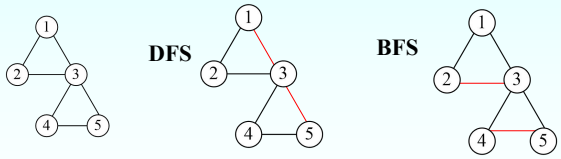
6.4 图的连通性

6.4.1 生成树

6.4.2 最小生成树

生成树的定义

➢ 设图G=(V,E)是一个连通图，当从图中任一顶点出发遍历G时，将边集E(G)分成两个集合A(G)和B(G)，其中A(G)是遍历时所经过的边的集合，B(G)是遍历时未经过的边的集合。显然，G1=(V,A)是图G的子图，称子图G1为连通图G的生成树。



➢ 定义:一个连通图的生成树是一个极小的连通子图，它包含图中的全部n个顶点，但只有足以构成一棵树的n－1条边。

生成树的例题

例：一无向图G（V，E），其中V（G）={1，2，3，4，5，6，7}，E（G）={（1，2），（1，3），（2，4），（2，5），（3，6），（3，7），（6，7），（5，1）}，对该图从顶点3开始进行遍历，去掉遍历中未走过的边，得一生成树G'（V，E'），V（G'）=V（G），E（G'）={（1，3），（3，6），（7，3），（1，2），（1，5），（2，4）}，则采用的遍历方法是_____。

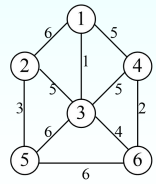
最小生成树

- 最小生成树：具有最小权的生成树。
- 得到最小生成树要解决好的两个问题：
  （1）尽可能选取权值小的边，但不能构成回路；
  （2）选取n－1条恰当的边以连接网的n个顶点。
- 最小生成树的MTS性质：
  假设N＝{V,E}是一个连通图，U是顶点集V的一个非空子集，若(u,v)是一条最小权值的边，其中u∈U，v∈V-U，则必存在一棵包含边（u，v）的最小生成树。
- 最小生成树的算法：
  (1)普里姆(Prim)算法　　(2)克鲁斯卡尔(Kruskual)算法

Prim算法

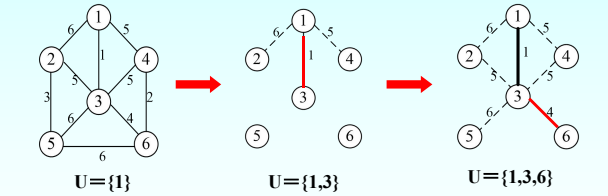
- 基本原理：从一个顶点U＝{u_0}作初态，不断寻找与U中顶点相邻且权值最小的边的另一个顶点，扩充到U集合直到U＝V为止。

例：试利用Prim算法求下图的最小生成树。

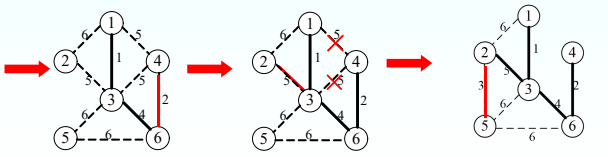


Prim算法的例题

假设以顶点1为开始顶点



U＝{1}　　U＝{1,3}　　U＝{1,3,6}

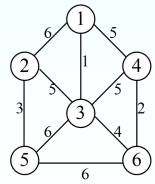


U＝{1,3,6,4}　　U＝{1,3,6,4,2}　　U＝{1,3,6,4,2,5}=V

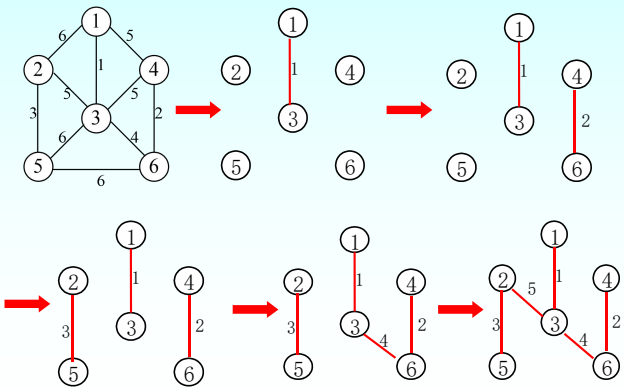
Kruskual算法

- 基本原理：假设连通网G=(V,E)，则令最小生成树的初始状态为只有n个顶点而无边的非连通图T=(V,{})，图中每个顶点自成一个连通分量。在E中选择代价最小的边，若该边依附的顶点落在T中不同的连通分量上，则将此边加入到T中，否则舍去此边而选择下一条代价最小的边。依次类推，直至T中所有顶点都在同一连通分量上为止。

例：试利用Kuskual算法求下图的最小生成树。

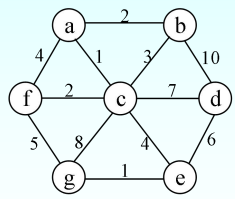


Kruskual算法的例题



最小生成树的例题

例：以顶点a为开始顶点，分别用Prim和Kruskual算法实现最小生成树

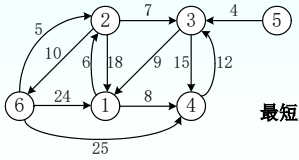


例：_____算法适用于求边稠密网的最小生成树；_____算法适用于求边稀疏网的最小生成树。

6.5 单源最短路径

最短路径的定义：如果从某顶点（源点）出发，经过图的边到达另一顶点（终点），所经过的路径不止一条，其中必存在一条路径，沿其路径上各边的权值之和为最小。

例：在下图中，若指定顶点v6为源点，试分析v6到其它各点的最短路径。



由v6→v1的路径：
v6→v1: 24
v6→v2→v1: 5+18=23
v6→v2→v3→v1: 5+7+9=21
v6→v4→v3→v1: 25+12+9=46

最短路径：v6→v1（经v2，v3）: 21
v6→v2: 5
v6→v3（经v2）: 12
v6→v4：25
v6→v5: 无路径

迪杰斯特拉(Dijkstra)算法

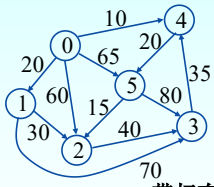
基本思想：

从图的给定源点到其它各个顶点之间客观上应存在一条最短路径，在这组最短路径中，按其长度的递增次序，依次求出到不同顶点的最短路径和路径长度。

即按长度递增的次序生成各顶点的最短路径，即先求出长度最小的一条最短路径，然后求出长度第二小的最短路径，依此类推，直到求出长度最长的最短路径。

迪杰斯特拉(Dijkstra)算法

例：试用Dijkstra算法求v0到其它各点的最短路径。



带权有向图及其邻接矩阵

$$\begin{pmatrix} \infty & 20 & 60 & \infty & 10 & 65 \\ \infty & \infty & 30 & 70 & \infty & \infty \\ \infty & \infty & \infty & 40 & \infty & \infty \\ \infty & \infty & \infty & \infty & 35 & \infty \\ \infty & \infty & \infty & \infty & \infty & 20 \\ \infty & \infty & 15 & 80 & \infty & \infty \end{pmatrix}$$

分析（设图中顶点数为n）
- As[1:n,1:n]—邻接矩阵，图中任意两点之间的权值（距离）。
- S—顶点集合，由源点到这些顶点的最短距离求出。
- Dist[1:n]—从源点到其它各点当前的最短距离。
- Pre[1:n]—从源点到某个顶点之间的最短路径的前驱顶点。

Dijkstra算法步骤

1) 初始化

2) 选择到源点最小距离的顶点
Dist[u]=min{Dist[w]|w∈S, w∈ V}

3) 调整各顶点到源点的距离，并修改相应的路径
Dist[w]=min{Dist[w],Dist[u]+As[u,w]}

4) 重复2)~3)，共n-1次（即S=V为止）

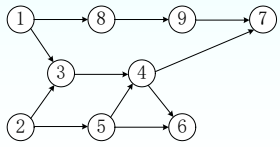
Dijkstra算法的实现过程

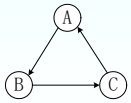
步骤	顶点	1	2	3	4	5	S
初态	Dist	20	60	∞	10	65	{0}
	pre	0	0	-1	0	0	
1	Dist	20	60	∞	10	30	{0, 4}
	pre	0	0	-1	0	4	
2	Dist	20	50	90	10	30	{0, 4, 1}
	pre	0	1	1	0	4	
3	Dist	20	45	90	10	30	{0, 4, 1, 5}
	pre	0	5	1	0	4	
4	Dist	20	45	85	10	30	{0, 4, 1, 5, 2}
	pre	0	5	2	0	4	
5	Dist	20	45	85	10	30	{0, 4, 1, 5, 2, 3}
	pre	0	5	2	0	4	

6.6 拓扑排序

- AOV网（activity on vertex network）的定义：
顶点表示活动、边表示活动间的先后关系的有向图称为顶点活动图，简称AOV图。

- AOV网是一个有向无环图，即不带有回路（因为若有回路，则回路上的所有活动都无法进行）。

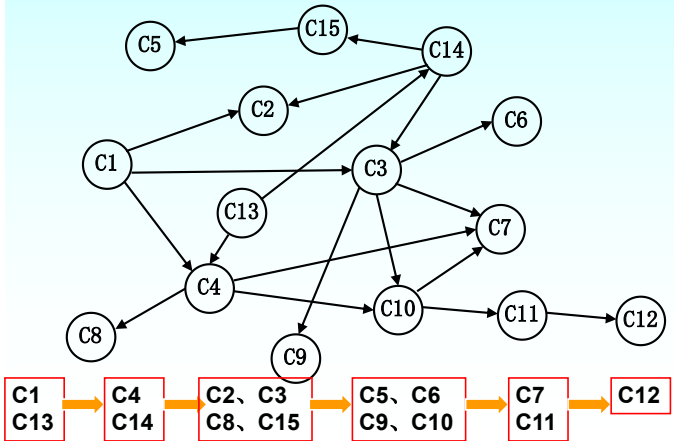




例：计算机专业的课程设置及其关系

课程编号	课程名称	先修课程	课程编号	课程名称	先修课程
C1	程序设计导论	无	C9	算法分析	C3
C2	数值分析	C1，C14	C10	高级语言	C3，C4
C3	数据结构	C1，C14	C11	编译系统	C10
C4	汇编语言	C1，C13	C12	操作系统	C11
C5	自动机理论	C15	C13	解析几何	无
C6	人工智能	C3	C14	微积分	C13
C7	计算机图形学	C3，C4，C10	C15	线性代数	C14
C8	计算机原理	C4			

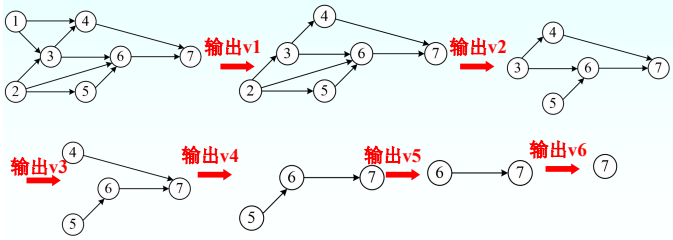
拓扑排序——将AOV网转换成表示优先关系的线性序列



拓扑排序的实现

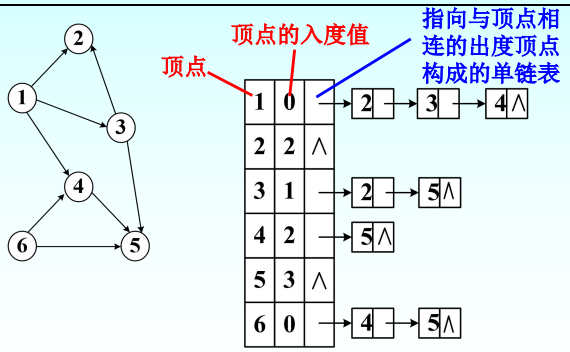
拓扑排序的步骤：

1) 在AOV网中选一个没有前驱（入度为0）的顶点并输出之。
2) 从AOV网中删除此顶点及其所有弧。
3) 重复上述两步，直至全部顶点都已输出，或图中没有入度为零的顶点为止。



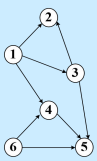
拓扑序列为：v1 → v2→ v3→ v4→ v5→ v6→ v7

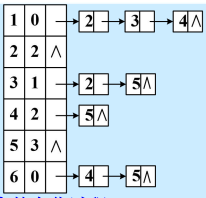
拓扑排序的邻接表



为了避免重复检测入度为零的顶点，另设一栈暂存所有入度为零的顶点。(算法参见书P182)

例：





顶点入度的变化过程

顶点	v1	v2	v3	v4	v5	v6
初态	0	2	1	2	3	0
v6出栈	0	2	1	1	2	0
v1出栈	0	1	0	0	2	0
v4出栈	0	1	0	0	1	0
v3出栈	0	0	0	0	0	0
v5出栈	0	0	0	0	0	0
V2出栈	0	0	0	0	0	0

栈的变化过程（入度为0的结点立刻进栈）

v1 v6	v1、v6入栈
v1	v6出栈
	v1出栈
v3 v4	v3、v4入栈
v3	v4出栈
	v3出栈
v2 v5	v2、v5入栈
v2	v5出栈
	v2出栈

使用栈的拓扑排序序列为：
v6、v1、v4、v3、v5、v2

改用队列，则输出的拓扑序列
v1, v6, v3, v4, v2, v5

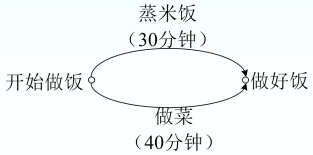
6.7关键路径

•AOE网（activity on edge network）的定义：
顶点表示事件、弧表示活动，弧上的权表示完成这一活动所需的时间，则称边活动网，简称AOE网。

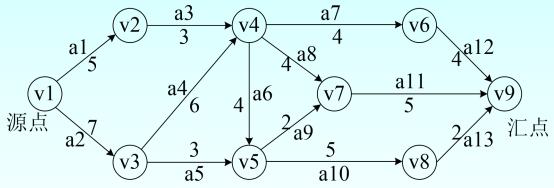
•特点：

1）AOE网用于估计工程的完成时间，所以没有回路；

2）存在唯一的入度为0的开始顶点，及存在唯一的出度为0的结束顶点。



AOE网的理解



上图为具有13个活动(子工程)a1,a2,…,a13的工程的AOE网,其中有9个顶点,分别表示事件(状态)v1到事件v9.

顶点：v1－工程开始；v2－活动a1完成；v4－活动a3、a4完成；v9－活动a12、a11、a13完成。

边：当工程开始后，活动a1、a2可以同时进行，而活动a3要等到事件v2发生后（即在活动a1完成后）才能进行，当活动a12、a11、a13完成后，整个工程就完成了。

关键路径的定义

AOE网所关心的问题：
1）完成一个工程至少需要多少时间？
2）哪些活动是影响整个工程进程的关键？

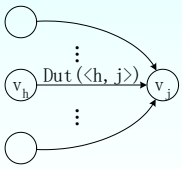
关键路径：从开始顶点到结束顶点的最大路径。关键路径的路径长度可以表示对应的工程所花费的最少时间。

关键活动：关键路径上的所有活动。

关键路径有关的量(1)

顶点事件vj的最早发生时间ve(j)
——从源点v1到vj的最长路径长度。这个时间决定了所有从vj发出的弧所表示的活动能够开工的最早时间

例：v1的最早发生时间ve（1）＝0；
v2的最早发生时间ve（2）＝5；
v3的最早发生时间ve（3）＝7；
v4的最早发生时间ve（4）
v1->v2->v4: 5+3=8
v1->v3->v4: 7+6=13 ✓



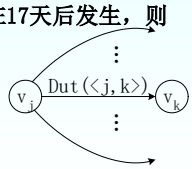
$$\begin{cases} ve(1)=0 \\ ve(j)=\max\{ve(h)+dut(<h,j>)\} \end{cases} \quad <h,j>\in T, 2\leqslant j\leqslant n$$

弧<h,j>上的权值　　所有以j为头的弧的集合

关键路径有关的量(2)

顶点事件vj的最迟发生时间vl(j)
——在不推迟整个工程完成日期的前提下，事件vj允许的最晚发生时间。
保证汇点vn在ve(n)时刻发生

例：若要保证v4最迟在13天后发生，则v2的最迟发生时间
vl（2）＝13－3＝10
若要保证v4最迟在13天后、v5最迟在17天后发生，则v3的最迟发生时间vl（3）
v4→v3:13-6=7
v5→v3:17-3＝14 ✓

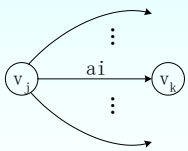


$$\begin{cases} vl(n)=ve(n) \\ vl(j)=\min\{vl(k)-dut(<j,k>)\} \end{cases} \quad <j,k>\in S, 1\leqslant j\leqslant n-1$$

弧<j,k>上的权值　　所有以j为尾的弧的集合

关键路径有关的量(3)

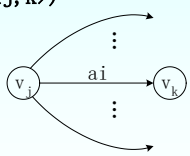
边活动ai的最早开始时间e(i)
——活动ai的最早开始时间等于事件vj的最早发生时间。
e（i）＝ve（j）



例：边活动a3的最早开始时间e(3)=ve(2)=5

关键路径有关的量(4)

边活动ai的最晚开始时间l(i)
——在不推迟整个工程完成时间的前提下，允许该活动ai的最晚开始时间。
l(i)=vl(k)-dut(<j,k>)



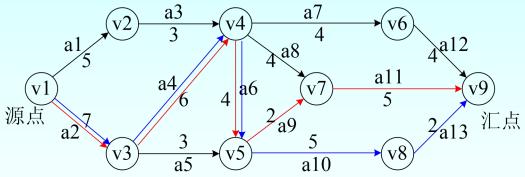
例：边活动a3的最迟开始时间
e(3)=vl(4)－dut(<2,4>)=13-3=10

关键路径有关的量(5)

对于边活动ai，若e(i)=l(i)，则表示该活动最早可开工时间与整个计划允许该活动最晚开始时间一致，施工期一点也不能拖延。

活动ai是关键活动{ ai不能按期完成→工程延期；ai提前完成→工程可能提前完工

关键路径的例题

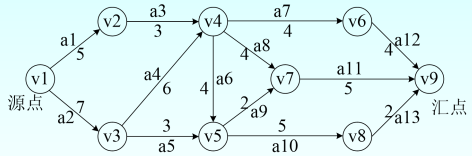


该AOE网的最少工程时间是24。
关键活动是a2，a4，a6，a9，a10，a11，a13。
关键路径是v1-v3-v4-v5-v7-v9和v1-v3-v4-v5-v8-v9。

如果网中存在多条关键路径，那么
（1）单是提高一条关键路径上的关键活动的速度，不能缩减整个工程的工期；
（2）必须同时提高几条关键路径上的活动速度。

关键路径的例题

例：试分析下图的关键路径。



步骤：

a.从ve(1)=0开始向前递推

$ve(j)=\max\{ve(h)+dut(<h,j>)\}$　　$<h,j>\in T, 2\leqslant j\leqslant n$

b. 从vl(n)=ve(n)起向后递推

$vl(j)=\min\{vl(k)-dut(<j,k>)\}$　　$<j,k>\in S, 1\leqslant j\leqslant n-1$

c. 根据各顶点的ve、vl值，求每个弧的e(i)和l(i)。若e(i)=l(i)，则为关键活动。

$e(i)=ve(j)$　$l(i)=vl(k)-dut(<j,k>)$